



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

SERVICIU GASTRONOMIC ONLINE CU APLICAȚII DIRECTE ACASĂ

Kookio

Absolvent: Eduard - Emanuel Dinea

Coordonator științific: Lect. dr. Mihai Cherciu

București, iulie 2026

Rezumat

Lucrarea de față documentează proiectarea și implementarea aplicației gastronomice *Kookio*, o platformă full-stack realizată integral în **TypeScript**. Aceasta combină avantajele unui meta-framework modern pentru **Angular 20** (Analog), ale unui strat backend type-safe construit cu **trPC** și ale unui ORM performant (**Prisma**) peste **PostgreSQL**, toate coordonate într-un monorepo gestionat de **Nx**.

Arhitectura **PAN** (Prisma – Analog – Nx) permite randare server-side, modularitate extinsă, testare automată și scalabilitate. Lucrarea detaliază deciziile tehnice, implementarea componentelor esențiale și direcțiile de dezvoltare viitoare, fiind însoțită de o anexă care descrie metodologia de redactare academică în **L^AT_EX**.

Cuprins

1	Introducere	3
2	Preliminarii	4
3	Implementare	6
3.1	Prezentare generală	6
3.2	Arhitectura sistemului	6
3.2.1	Structura monorepo-ului Nx	6
3.2.2	Flux aplicație end-to-end	6
4	Concluzii	7
	Appendices	8
A	Anexă: Redactarea lucrării în L^AT_EX	9
A.1	Instrumente pentru redactare	9
A.2	Compatibilitate multiplatformă	9
B	Kookio: Manifest Arhitectural PAN (Prisma – Analog – Nx)	10
B.1	Viziune generală	10
B.2	Componentele arhitecturii	10
B.3	Structura fizică a proiectului	10
B.4	Avantaje comparative	11
B.5	Dir ecții de viitor	11
B.6	Concluzie	11

Capitolul 1

Introducere

Motivație

În contextul vieții moderne, tot mai multe persoane caută soluții eficiente pentru organizarea meselor zilnice. Aplicațiile gastronomice devin tot mai relevante atunci când oferă recomandări personalizate, gestionare a ingredientelor și integrare cu surse externe de date. Proiectul *Kookio* răspunde acestor nevoi printr-o arhitectură modernă PAN, care valorifică progresul tehnologic din ecosistemul TypeScript și permite dezvoltarea unei aplicații scalabile, testabile și ușor de întreținut.

Scopul lucrării

Lucrarea urmărește dezvoltarea unei aplicații web complete care să permită utilizatorilor:

- gestionarea ingredientelor personale,
- descoperirea de rețete în funcție de preferințe și disponibilitate,
- localizarea furnizorilor pentru produsele lipsă.

Prin urmare, *Kookio* propune o abordare end-to-end, de la interfață la baza de date, fundamentată pe principiile arhitecturii PAN. Pentru detalii de implementare, vezi Capitolul [3](#).

Capitolul 2

Preliminarii

Contextul tehnologic

Proiectul *Kookio* se bazează pe arhitectura unitară **PAN** (Prisma–Analog–Nx), care integrează fluxul complet de dezvoltare **TypeScript** de la interfață până la bazele de date. Această arhitectură facilitează modularitatea, testarea automată și extinderea sistemului. O prezentare detaliată a manifestului, a avantajelor comparative și a direcțiilor de evoluție este centralizată în **Anexa B**.

Tehnologii fundamentale

- **Analog** — meta-framework pentru *Angular*, oferă randare pe server (SSR), rutare pe bază de fișiere și integrare nativă cu Nitro/tRPC pentru definirea de API-uri[7, 11]. Componentele folosesc modelul de reactivitate *Signals* al *Angular*[8].
- **Angular** — framework modern ce susține *standalone components* și *Signals* pentru gestionarea stării în mod predictibil și performant[8].
- **Prisma** — ORM type-safe pentru PostgreSQL, cu migrații și generator de client TypeScript, alături de generatoare pentru validări (Zod)[16, 4]. Persistența se bazează pe mecanismele *ACID* ale *PostgreSQL*[15].
- **tRPC** — comunicare end-to-end *type-safe* între client și server, cu rutare, context și inferență de tipuri pe întreg lanțul aplicației[17].
- **Nx** — orchestrator monorepo care coordonează aplicații și biblioteci, oferind *affected runs*, cache distribuit și graful de proiect pentru optimizări de build și test[10].
- **Testare** — *Vitest* pentru testare unitară rapidă[13] și *Playwright* pentru testare end-to-end[9].

Principii arhitecturale

- **Separarea responsabilităților** — interfață (Analog/Angular), logică de domeniu (tRPC + modele), acces la date (Prisma)[7, 8, 17, 16].
- **Type-safety end-to-end** — contracte de tipuri partajate între client și server (tRPC) și validări derivate din modele (Prisma → Zod)[17, 16, 4, 2].
- **Modularitate monorepo** — organizare pe aplicații și librării, reguli de dependență, execuții afectate și cache incremental (Nx)[10].
- **Randare hibridă și API-uri** — SSR prin Analog, rute server și integrare cu mediile de execuție prin Nitro[7, 11].
- **Observabilitate și configurare** — logare centralizată (Pino) și protecție la configurări incomplete (dotenv-safe)[1, 6].

Capitolul 3

Implementare

3.1 Prezentare generală

Aplicația *Kookio* a fost dezvoltată conform arhitecturii **PAN** descrise în **Anexa B**. Aceasta integrează tehnologii moderne din ecosistemul **TypeScript** pentru a asigura un flux end-to-end unitar între frontend și backend. În cele ce urmează sunt prezentate componentele interne ale sistemului, fluxurile principale și mecanismele de automatizare.

3.2 Arhitectura sistemului

3.2.1 Structura monorepo-ului Nx

```
kookio/
├── web/                                # aplicația Analog
│   └── src/                            (frontend + server SSR)
│       ├── app/                       # componente Angular, pagini, servicii
│       └── server/                    # rute Nitro, tRPC și handler-e API
├── libs/
│   ├── data-access/prisma/           # schema Prisma, migrare,
│   │                                   client, validări Zod
│   └── shared/                       # mock-uri și utilitare comune
└── nx.json, package.json, etc.      # configurarea monorepo-ului Nx
```

3.2.2 Flux aplicație end-to-end

Fluxul aplicației urmează schema: *Analog (UI/SSR)* → *tRPC (API type-safe)* → *Prisma (bază de date)* → *Nx (orchestrare)*. Acest flux permite un lanț complet de tipuri, logare unitară și construcție incrementală a proiectului.

Capitolul 4

Concluzii

Lucrarea demonstrează că arhitectura **PAN** permite o integrare coerentă a front-end-ului Angular (via Analog), a stratului API (Nitro + tRPC) și a persistenței (Prisma/PostgreSQL), sub orchestrarea **Nx**. Implementarea *Kookio* confirmă beneficii de productivitate (tipare re-utilizabile, *type-safety* end-to-end), scalabilitate (monorepo, cache, „affected runs”) și fiabilitate (validări Zod, logger comun).

Avantajele comparative sunt sintetizate în Tabelul [B.1](#), iar justificarea arhitecturală detaliată este centralizată în Capitolul [B](#). În ansamblu, abordarea PAN oferă o bază solidă pentru produse web moderne, extensibile și ușor de întreținut.

Direcții viitoare

- CockroachDB și Redis pentru scalare orizontală și reducerea latențelor.
- Autentificare Firebase Auth și politici de autorizare bazate pe roluri.
- Export de JSON Schema pentru interoperabilitate cu parteneri.
- Consolidarea E2E (Playwright) în pipeline „affected”.
- Microserviciu AI pentru recomandări personalizate (Nitro → Analog).

Appendices

Anexa A

Anexă: Redactarea lucrării în L^AT_EX

A.1 Instrumente pentru redactare

Lucrarea a fost redactată cu motorul LuaLaTeX, folosind fonturi Unicode și pachetul `biblatex` pentru gestionarea bibliografiei. Structura este modulară, fiecare capitol fiind un fișier separat (`.tex`) compilat prin scripturile automate de build. Imaginile sunt salvate în `images/`, iar citările se realizează cu `\cite{}`.

A.2 Compatibilitate multiplatformă

Există scripturi automate pentru Linux (`build.sh`), Windows (`build.bat`) și PowerShell (`build.ps1`) care generează PDF-ul în directorul `dist/` și afișează eventualele erori de compilare.

Notă: Mediul `TreeVerb` utilizează fontul `FreeMono` pentru suport complet al caracterelor Unicode de tip box-drawing (U+2500–U+257F).

Anexa B

Kookio: Manifest Arhitectural PAN (Prisma – Analog – Nx)

B.1 Viziune generală

Kookio este o aplicație full-stack dezvoltată în **TypeScript**, bazată pe arhitectura **PAN** — un cadru scalabil care integrează **Prisma** pentru acces la date, **Analog** pentru randare hibridă și **Nx** pentru orchestrare monorepo. Această abordare reprezintă o alternativă la stack-uri consacrate (JAMstack, MEAN), fiind adaptată ecosistemului **Angular** și cerințelor enterprise.

B.2 Componentele arhitecturii

- **Prisma** – ORM cu suport pentru PostgreSQL și extensii NewSQL (CockroachDB).
- **Analog** – meta-framework pentru Angular cu SSR, Signals și tRPC.
- **Nx** – orchestrator modular care optimizează build-urile și testele.

B.3 Structura fizică a proiectului

Structura completă a monorepo-ului este ilustrată în § 3.2.1 din capitolul 3.

B.4 Avantaje comparative

Caracteristică	JAMstack	MEAN	PAN (Kookio)
SSR / Hybrid Rendering	Parțial (Next.js)	Nu	Da (Analog SSR)
Type-safety end-to-end	Parțial	Nu (Mongo/JS)	Da (TS + Prisma + Angular)
Structurare modulară	Slabă	Medie	Excelentă (Nx + libs)
Acces la date	Headless CMS/REST	REST/Mongoose	ORM modern (Prisma)
Adaptabilitate mono-repo	Nu	Nu	Da

Tabela B.1: Comparatie între arhitectura PAN și alte stack-uri moderne

B.5 Direcții de viitor

Direcțiile de dezvoltare planificate sunt detaliate în capitolul § 4.

B.6 Concluzie

Arhitectura **PAN** sintetizează cele mai bune practici moderne din ecosistemul **TypeScript**, oferind o experiență unificată full-stack și o bază solidă pentru aplicații educaționale, alimentare sau de lifestyle, precum viziunea *Kookio*.

Bibliografie

- [1] Autorii Pino, *Pino — Logger pentru Node.js*, Documentația Pino, Documentație oficială Pino; citată pentru logarea centralizată utilizată atât în backend, cât și în frontend., URL: <https://getpino.io/>.
- [2] Autorii Zod, *Documentația Zod*, Documentația Zod, Documentație oficială Zod; citată pentru validarea schemelor utilizate împreună cu tipurile generate de Prisma., URL: <https://zod.dev>.
- [3] Cockroach Labs, *Documentația CockroachDB*, Documentația CockroachDB, Documentație oficială CockroachDB; citată în planul de dezvoltare pentru analiza scalabilității orizontale., URL: <https://www.cockroachlabs.com/docs/stable/>.
- [4] Omar Dulaimi și colaboratorii, *Generatorul Prisma Zod*, Depozitul GitHub, Versiunea 1.27.7; citată pentru generarea automată a schemelor de validare Zod pe baza modelelor Prisma., URL: <https://github.com/omar-dulaimi/prisma-zod-generator>.
- [5] Comunitatea JSON Schema, *JSON Schema — Învăță (Learn)*, Documentația JSON Schema, Resursă oficială JSON Schema; citată pentru validarea externă și interope-rabilitatea menționată în planul de dezvoltare., URL: <https://json-schema.org/learn/>.
- [6] Comunitatea Motdotla, *dotenv-safe — Validarea variabilelor de mediu necesare*, Depozitul GitHub, Utilizat pentru a asigura prezența variabilelor de mediu obligatorii la rulare; completează logarea centralizată implementată cu Pino., URL: <https://github.com/rolodato/dotenv-safe>.
- [7] Dezvoltatorii AnalogJS, *Analog – Meta-framework full-stack pentru Angular*, Docu-mentația Analog, Documentație oficială Analog; citată pentru SSR, rutare bazată pe fișiere și integrarea Nitro/tRPC în aplicațiile Angular., URL: <https://analogjs.org>.
- [8] Echipa Google Angular, *Prezentare generală a semnalelor (Signals Overview)*, Docu-mentația Angular, Documentație oficială Angular; citată pentru modelul de reactivi-tate (Signals) utilizat în componente independente și în contextul randării pe server (SSR)., URL: <https://angular.dev/guide/signals>.
- [9] Echipa Microsoft Playwright, *Testarea cu Playwright — Ghid introductiv*, Documen-tația Playwright, Documentație oficială Playwright; citată pentru testarea end-to-end adoptată în perioada iunie–iulie 2025., URL: <https://playwright.dev/docs/intro>.
- [10] Echipa Nrwl, *Ce este Nx?*, Documentația Nx, Documentație oficială Nx; citată pentru orchestrarea monorepo-ului, rulările afectate, caching și graful de proiect., URL: <https://nx.dev/getting-started/intro>.

-
- [11] Echipa UnJS, *Nitro: Motorul de server pentru web*, Documentația Nitro, Documentație oficială Nitro; citată pentru execuția pe server, rutele API și mediile de implementare compatibile cu Analog SSR., URL: <https://nitro.unjs.io>.
 - [12] Echipa Vite, *Ghidul Vite*, Documentația Vite, Documentație oficială Vite; citată pentru configurarea build-ului comună aplicațiilor și bibliotecilor și integrarea cu țintele Nx., URL: <https://vitejs.dev/guide/>.
 - [13] Echipa Vitest, *Ghidul Vitest*, Documentația Vitest, Documentație oficială Vitest; citată pentru configurarea și utilizarea testelor unitare, folosită în proiect începând cu august 2025., URL: <https://vitest.dev/guide/>.
 - [14] Google Firebase, *Autentificarea în Firebase*, Documentația Firebase, Documentație oficială Firebase; citată pentru autentificarea planificată și politicile de autorizare bazate pe roluri., URL: <https://firebase.google.com/docs/auth>.
 - [15] Grupul Global de Dezvoltare PostgreSQL, *Documentația oficială PostgreSQL*, Documentația PostgreSQL, Documentație oficială PostgreSQL; citată pentru semantica SQL, suportul JSONB și siguranța tranzacțiilor utilizate prin Prisma., URL: <https://www.postgresql.org/docs/>.
 - [16] Prisma Labs, *Documentația Prisma ORM*, Documentația Prisma, Documentație oficială Prisma; citată pentru modelarea schemei, migrații, generarea clientului TypeScript și integrarea cu Zod., URL: <https://www.prisma.io/docs>.
 - [17] Proiectul tRPC, *Documentația tRPC — API-uri type-safe end-to-end*, Documentația tRPC, Documentație oficială tRPC; citată pentru contractele de tipuri între client și server și pentru structura router/context., URL: <https://trpc.io/docs>.
 - [18] Redis, *Documentația Redis*, Documentația Redis, Documentație oficială Redis; citată în planul de dezvoltare pentru strategiile de caching și gestionare a sesiunilor de interogare., URL: <https://redis.io/docs/latest/>.